AI ENGINEERING PROJECT

Insurance Policy Intelligence AI

A production-oriented RAG platform for insurance agents

PROJECT VISION
Help insurance agents understand policy documents, compare coverage, and answer customer questions with traceable evidence—without manually searching through lengthy PDFs.

Project snapshot

Category	AI Engineering / Document Intelligence / SaaS
Primary user	Insurance agents and policy advisors
Core capability	RAG-based question answering with source citations
MVP outcome	Upload a policy PDF → ask questions → receive evidence-backed answers
Future product	Policy comparison, customer CRM, renewal workflow

Portfolio positioning

Build more than a chatbot: demonstrate document ingestion, hybrid retrieval, reranking, grounded generation, evaluation, security, observability, and deployment in one practical domain.

Prepared as a project planning brief • October 2026

1. Problem statement and solution

Problem: Insurance policies contain dense clauses, exclusions, waiting periods, coverage limits, and claim requirements. Agents may manage multiple products and insurers, making it difficult to find the right clause quickly and explain it consistently.

Solution: A web application where agents upload policy documents and ask natural-language questions. The system retrieves relevant passages, generates a concise explanation, and returns the document, page, and section that support the answer.

Example user question

> **“Does this policy cover a surgery after six months?”**

The AI should retrieve the applicable waiting-period clause and explain that coverage depends on the policy wording, condition, and insurer decision. It must not invent coverage or present itself as the final claims authority.

2. Product scope

MVP features

- **Policy upload:** Upload PDF policies and store document metadata.
- **Policy Q&A;:** Ask questions about coverage, exclusions, waiting periods, limits, and claim requirements.
- **Evidence-backed answers:** Show source title, page number, section, and retrieved text.
- **Policy library:** Search and select documents before asking questions.
- **Unknown-answer handling:** Clearly say when the document does not contain enough evidence.

Phase 2 features

- **Policy comparison:** Compare coverage, exclusions, waiting periods, limits, and eligibility conditions across policies.
- **Customer and policy CRM:** Associate customers with policies and store renewal dates.
- **Renewal assistant:** Show upcoming expiries and draft reminders.
- **Customer-friendly summaries:** Generate simplified explanations for agent review.

Out of scope for the first MVP

- Automated claim approval or definitive coverage decisions.
- Training a foundation model from scratch.
- Complex multi-agent systems before baseline RAG works.
- Processing confidential real customer data without approved security and consent controls.

3. Recommended technology stack

Layer	Technology	Why it fits
Frontend	Next.js + TypeScript	Responsive dashboard, upload, chat, policy comparison
UI	Tailwind CSS + shadcn/ui	Fast, consistent UI components
Backend	Python + FastAPI + Pydantic	Typed APIs and convenient AI/document ecosystem
LLM	OpenAI API	Answer generation, extraction, structured output

Layer	Technology	Why it fits
Database	PostgreSQL + pgvector	Relational CRM data plus vector search in one database
Document parsing	PyMuPDF; Docling or Unstructured as needed	PDF text/layout extraction; OCR fallback for scans
Retrieval	BM25 + vector similarity + fusion	Keyword and semantic retrieval complement each other
Reranking	Cross-encoder or hosted reranker	Reorder candidates by relevance before generation
Orchestration	Plain Python first; LangGraph later	Keep baseline simple; add controlled multi-step flow when useful
Jobs	Background worker; Redis/Celery if needed	Process larger uploads without blocking API requests
Observability	Langfuse	Trace prompts, retrieval, latency, errors, and usage
Evaluation	RAGAS + custom test set	Measure retrieval and answer quality; avoid relying on one score
Deployment	Docker + CI workflow	Repeatable builds, tests, and deployment

Stack principle: Use established components and measure their value. Do not add a framework just because it is trending. Start with PostgreSQL/pgvector and a clear pipeline; introduce orchestration, caching, or extra infrastructure only when the workload requires it.

4. System architecture

Request flow

Stage	Description
1. Upload	Agent uploads policy PDF through Next.js to FastAPI.
2. Ingest	Validate file, extract text/layout, OCR scanned pages if necessary.
3. Chunk	Split by headings/clauses; preserve page, section, policy, and insurer metadata.
4. Embed	Generate embeddings and store chunks plus vectors in PostgreSQL/pgvector.
5. Retrieve	Run metadata filters, keyword search, vector search, and merge candidates.
6. Rerank	Reorder candidates; select the most relevant evidence.
7. Generate	LLM answers using retrieved context and strict grounding instructions.
8. Validate	Check citation IDs and source metadata; abstain if evidence is insufficient.
9. Return	Display answer, source snippets, page references, and limitations.

Core design rules

- Every chunk must retain document ID, policy ID, page number, section, and tenant ID.
- Retrieval must filter by the user's authorized tenant and selected policy set.
- The LLM must treat uploaded policy text as untrusted content, not as instructions.
- Citations must point to retrieved source chunks—not be generated as free-form page guesses.
- If evidence is missing or conflicting, explain the uncertainty and ask for clarification.

5. RAG engineering plan

Baseline pipeline

- Parse PDF text and preserve page boundaries.
- Chunk by headings and policy clauses rather than arbitrary character windows alone.
- Generate embeddings and store them with source metadata.
- Retrieve top-k chunks using vector similarity.
- Construct a grounded prompt with source IDs and text.
- Generate an answer and render verified citations.

Advanced retrieval experiments

Experiment	What to compare
Vector search	Embedding similarity baseline
BM25	Keyword-only retrieval for exact terms and clause names
Hybrid retrieval	Combine keyword and vector candidate lists
Reranking	Compare ranking quality after reranker
Query rewriting	Test whether rewritten queries improve recall
Corrective retrieval	Retry only when evidence quality is insufficient; cap retries

Recommended progression

Vector baseline → hybrid retrieval → reranking → controlled query rewrite/retry. Keep the simplest version that meets the evaluation target. Add LangGraph only when you need explicit state, branching, retries, or multi-step comparison.

6. Evaluation and quality metrics

Create a small, manually verified evaluation set before tuning the pipeline. For each question, record the expected answer, supporting policy, page/section, and whether it is answerable.

Metric	Purpose
Recall@k	Did retrieval include the expected evidence in the top-k results?
MRR / nDCG	Was the relevant evidence ranked near the top?
Answer correctness	Does the answer match the policy evidence and expected answer?
Groundedness	Are claims supported by retrieved text?
Citation accuracy	Do citations point to the right document/page/section?
Abstention quality	Does the system refuse to guess when evidence is missing?
Latency and cost	Track response time and API usage per question

Initial benchmark target (proposed, not measured)

Use a 50–100 question test set as an initial milestone. Set targets after establishing a baseline—for example, improve evidence Recall@5 and citation accuracy across successive retrieval experiments. Report actual results, including failures, rather than claiming unmeasured accuracy.

7. Six-week build roadmap

Week	Focus	Deliverable
1	Foundation + ingestion	Repo, API, DB, PDF extraction, chunk metadata

Week	Focus	Deliverable
2	Baseline RAG	Embeddings, vector retrieval, chat, source citations
3	Retrieval engineering	BM25 + vector hybrid, reranking, benchmark set
4	Structured workflows	Policy comparison, structured extraction, bounded retry
5	Evaluation + security	Metrics, tenant isolation, prompt-injection tests, abstention
6	Deployment + portfolio	Docker, CI, demo, README, architecture and evaluation report

Weekly working cadence

- Weekdays: implement one small component, write tests, and document decisions.
- Weekend: integrate components, run evaluation, review errors, and plan the next experiment.
- Each week should end with a demonstrable milestone—not just completed tutorials.

8. Initial API and data model

Endpoint	Purpose
POST /policies/upload	Upload and register a policy document
POST /policies/{id}/process	Trigger or inspect ingestion
GET /policies	List policies accessible to the tenant
POST /chat	Ask a question scoped to authorized policies
POST /policies/compare	Compare selected policies using retrieved evidence
GET /customers	List authorized customers (Phase 2)
GET /renewals	List upcoming renewals (Phase 2)

Core entities

- **Tenant:** agent organization and access boundary.
- **User:** authenticated agent or staff member.
- **Policy:** insurer, product, policy version, metadata.
- **Document:** file location, processing status, checksum.
- **Chunk:** text, embedding, page, section, policy, tenant.
- **Customer / CustomerPolicy:** CRM association and renewal date (Phase 2).
- **Conversation / Citation:** question, answer, retrieved chunk IDs, trace metadata.

9. Security, reliability, and responsible use

- Tenant isolation: enforce authorization at API and database query layers.
- Document safety: treat PDFs and extracted text as untrusted; ignore embedded instructions.
- Privacy: use public or synthetic policy data for portfolio demos; avoid exposing customer PII.
- File handling: validate MIME/type, size limits, malware scanning where available, and safe storage.
- Grounding: require source-backed answers; make uncertainty explicit.
- Versioning: identify policy versions and avoid mixing clauses from different products.

- Human review: agent verifies explanations before sharing with customers.
- Auditability: log retrieval IDs, model version, prompt version, and evaluation outcomes without logging unnecessary sensitive content.

Main risks

Risk	Mitigation
Wrong clause retrieved	Hybrid retrieval, reranking, filters, evaluation
Invented coverage answer	Grounded prompt, citations, abstention, human review
Conflicting policy versions	Version metadata and explicit document selection
Prompt injection in PDF	Treat source text as data; never follow its instructions
Sensitive customer data exposure	Tenant isolation, access control, approved storage and retention

10. Repository structure and portfolio deliverables

Suggested repository

insurance-intelligence-ai/ ■■■ apps/web/ # Next.js dashboard ■■■ services/api/ # FastAPI routes and auth ■■■ services/ingestion-worker/ # PDF parsing and indexing ■■■ rag/ ■ ■■■ ingestion/ ■ ■■■ chunking/ ■ ■■■ embeddings/ ■ ■■■ retrieval/ ■ ■■■ reranking/ ■ ■■■ generation/ ■ ■■■ evaluation/ ■■■ tests/ # retrieval, generation, security ■■■ evals/ # verified questions and expected evidence ■■■ infrastructure/ # Docker and CI ■■■ docs/ # architecture, experiments, limitations

Portfolio deliverables

- Live demo with sample policy documents.
- Short walkthrough video showing upload, question, answer, and citations.
- Architecture diagram and setup instructions.
- Retrieval experiment table with actual metrics.
- Security and limitations document.
- A clear README explaining the engineering decisions.

Suggested portfolio description

Built an insurance policy intelligence platform using Next.js, FastAPI, PostgreSQL/pgvector, and OpenAI. Implemented PDF ingestion, metadata-aware chunking, hybrid retrieval, reranking, grounded answer generation, source citations, and an evaluation suite for retrieval quality, answer correctness, and citation accuracy. Designed the system with tenant isolation, document versioning, and safe abstention when evidence is insufficient.

11. Recommended build order

- **First:** Upload one public policy PDF and extract text with page metadata.
- **Second:** Implement vector retrieval and answer generation with citations.
- **Third:** Build a 50-question benchmark and measure baseline quality.
- **Fourth:** Add hybrid search and reranking; compare against baseline.
- **Fifth:** Add policy comparison and controlled corrective retrieval.
- **Sixth:** Add auth, tenant isolation, tracing, deployment, and portfolio documentation.
- **Later:** Add CRM and renewal reminders after the AI core is reliable.

Success criterion

A deployed application that answers policy questions using retrieved evidence, cites the correct source, handles unknowns safely, and includes a reproducible evaluation report.

Note: This brief is a proposed engineering plan, not insurance advice or a claim of measured system performance. Validate all policy answers against the source wording and relevant insurer terms.